

Kubernetes Orchestration & Helm

Category: **Container Orchestration & Cloud Native Platforms** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Kubernetes Orchestration & Helm**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- **Control Plane Architecture & etcd State Consistency:** The control plane consists of kube-apiserver (REST API gateway), etcd (distributed consensus key-value store using Raft), kube-scheduler (assigns pods to nodes based on constraints), and kube-controller-manager (reconciliation loops).

Production Application: etcd latency directly dictates cluster performance. SREs must monitor fsync disk latency and maintain automated etcd snapshots before performing cluster upgrades.

- **Pod Lifecycle & Probe Mechanics (Startup, Liveness, Readiness):** Startup probes protect slow-starting legacy apps; Liveness probes kill and restart unresponsive containers; Readiness probes remove unhealthy pods from Service endpoints so they do not receive user traffic.

Production Application: Miscalibrated Liveness probes that point to heavy database queries cause cascading outages: under high load, queries slow down, probes fail, and Kubernetes restarts pods simultaneously.

- **Resource Quality of Service (QoS) Classes: Guaranteed, Burstable, BestEffort:** Guaranteed pods have equal CPU/memory requests and limits. Burstable pods have requests lower than limits. BestEffort pods have no requests or limits specified.

Production Application: When a node encounters memory pressure, the kubelet evicts BestEffort pods first, then Burstable pods exceeding requests, protecting Guaranteed mission-critical workloads.

- **Helm 3 Architecture & Release Lifecycle:** Package manager for Kubernetes using client-side templating (Go templates) and storing release state directly as Kubernetes Secrets within the target namespace, eliminating Tiller security vulnerabilities.

Production Application: Enterprise teams manage environment configurations (dev, staging, prod) via separate `values.yaml` files, enabling automated atomic rollouts with rollback capabilities.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
kubectl describe pod -n kubectl describe pod -n	Inspect detailed lifecycle state, resource allocations, probe definitions, and chronologically sorted cluster events for a pod. Options: [{flag: '-n ', 'description': 'Target Kubernetes namespace.'}, {flag: '-l ', 'description': 'Filter by label selector.'}]	Inspect the bottom 'Events' table to identify why a pod failed (e.g. FailedScheduling, Back-off pulling image, OOMKilled). Pitfall: Checking only <code>`kubectl get pods`</code> which only shows high-level status without the underlying event messages.
kubectl logs -f deployment/payment-service -n prod -c app-container --previous kubectl logs [options] [-c]	Stream logs from the previously crashed container instance within a deployment to determine root cause of termination. Options: [{flag: '-f', 'description': 'Follow log stream in real time.'}, {flag: '-c ', 'description': 'Specify container name in multi-container pods.'}, {flag: '--previous', 'description': 'Retrieve logs from the previous terminated instance of the container.'}, {flag: '--tail=100', 'description': 'Display only the most recent 100 log lines.'}]	Execute with <code>`--previous`</code> immediately after a container restarts to catch fatal exception stack traces before they are overwritten. Pitfall: Running <code>`kubectl logs`</code> without <code>`--previous`</code> on a restarting pod, which only returns logs from the newly spawned instance that may not have failed yet.
kubectl rollout undo deployment/payment-service -n prod --to-revision=2 kubectl rollout undo [options]	Perform an emergency instant rollback of a Kubernetes deployment to a known healthy historical revision. Options: [{flag: '--to-revision=', 'description': 'Specify exact historical revision number to rollback to.'}, {flag: '--dry-run=client', 'description': 'Simulate rollback without applying changes to cluster.'}]	Execute immediately upon detecting post-deployment error spikes to restore service while engineers debug in staging. Pitfall: Rolling back a deployment without checking whether database schema migrations are backwards-compatible.
kubectl drain node-worker-03 --ignore-daemonsets --delete-emptydir-data --force kubectl drain [options]	Safely evict all active workloads from a worker node in preparation for maintenance, kernel upgrades, or decommissioning. Options: [{flag: '--ignore-daemonsets', 'description': 'Bypass daemonset pods (which cannot be drained since they run on every node).'}, {flag: '--delete-emptydir-data', 'description': 'Acknowledge and proceed with deletion of ephemeral emptyDir data.'}, {flag: '--force', 'description': 'Force eviction of standalone pods not managed by a Deployment/StatefulSet.'}]	Cordon the node first, then drain, allowing PodDisruptionBudgets to maintain minimum available application replicas. Pitfall: Draining a node when workloads lack PodDisruptionBudgets (PDBs), leading to simultaneous termination of all service replicas.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
helm upgrade --install payment-gateway ./charts/payment -f values-production.yaml -n prod --atomic --timeout 5m helm upgrade --install [options]	Atomically install or upgrade a Helm release, rolling back automatically to previous revision if the upgrade fails within 5 minutes. Options: [{ 'flag': '--install', 'description': 'Install chart if release does not already exist.' }, { 'flag': '-f ', 'description': 'Override default values with environment-specific configuration.' }, { 'flag': '--atomic', 'description': 'Automatically purge or rollback changes if the upgrade process fails.' }, { 'flag': '--timeout 5m', 'description': 'Time limit to wait for all resources (pods, services) to reach ready state.' }]	Run in CI/CD deployment jobs with <code>--atomic</code> to ensure clusters never remain in a half-broken or inconsistent deployment state. Pitfall: Omitting <code>--atomic</code> and <code>--timeout</code> , allowing broken releases to remain partially applied with pods failing silently.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.